CODE REVIEW

Generate a unique numeric identifier for each instance of a class

Asked 5 years, 3 months ago Modified 3 years, 2 months ago Viewed 22k times

10

I want to generate a **unique ID** for every instance of a given class.

- The ID is of type `unsigned long`
- When an instance is deleted, the ID is freed (can be reused)

Now, I've come up with the following implementation:

```
Class Foo
{
public:

  static unsigned std::set<unsigned long> s_usedID;

  static unsigned long generateID() // Generate a valid ID
  {
    static unsigned long id = 0;

     while (Foo::isIDUsed(id)) // If all ID are taken, create an infinite loop!
        ++id;

     return id;
  }

  static void addID(unsigned long id) // Add a given ID to the set of used ID
  {
    s_usedID.insert(id);
  }

  static void removeID(unsigned long id) // Remove a given ID from the set of used ID
  {
    s_usedID.erase(id);
  }

  static bool isIDUsed(unsigned long id)
  {
    return s_usedID.count(id) == 1 ? true:false;
  }

  explicit Foo() : m_id(Foo::generateID())
  {
    Foo::addID(m_id); // ID is now taken
  }

  virtual ~Foo()
  {
    Foo::removeID(m_id); // Free the ID
  }

private:

  unsigned long m_id;
```

```
};
```

Now for me, this logically works.

- Is there a better way to implement this?
- Are there any optimizations that I could make?

c++ object-oriented

Share Improve this question Follow

edited Sep 11, 2017 at 0:04
Jamal
34.8k 13 131 235

asked Aug 22, 2017 at 5:08
Olivier
109 1 1 6

1 Not a C++ expert but it seems to me the infinite loop is ineffcient if large numbers of integers are taken. Also, the set `s_userID` could grow quite large if many unique IDs are used. Both of these things would cause me personally to rethink this algorithm. – markspace Aug 22, 2017 at 5:19

3 Is ID reusability an actual requirement? unsigned longs are big enough that you'll probably never run out of them unless you have google-sized data (in which case you can just make a custom longer int). – Rchn Aug 22, 2017 at 9:31

1 Do you have some sort of persistency, or should IDs be unique only in one run (so 2 runs with IDs (1, 2, 3) and (1, 2, 3) are ok)? Also, do you need to get all used IDs? – Daerdemandt Aug 22, 2017 at 9:43

1 @Olivier Not sure if this matters to you but note that this is currently not thread safe... so Ids could be reused if the class is used from multiple threads – Milney Aug 22, 2017 at 10:06

2 Why so complicated? Why not simple 64 bit counter? – Nick Aug 22, 2017 at 14:45

6 Answers

Sorted by: Highest score (default)

23

A few remarks:

1. The biggest obvious issue is the violation of encapsulation. All methods which allow you to modify the id set are public which means that outside code can change it and break the inherent assumption that each instance will have a unique id.

   The only code which should be able to modify the id set are the constructor and destructor.

   If you now say "ah, that won't matter it's just my code and I know not to do this" - programming is a skill and like any skill you get better at it through practice. Which typically means writing code. Therefore you should take any opportunity to practicing writing good code since that helps to form good habits. There is no excuse for being sloppy.

2. This:

```
return s_usedID.count(id) == 1 ? true:false
```

can be written more succinctly as:

```
return s_usedID.count(id) == 1;
```

3. The implementation is obviously not thread-safe. Just something to be aware of.
4. One way to improve it, is to just use a variable to store the current maximum id and return the next one higher if you need a new one. If an id gets freed up then you could just not worry about it. If you really want to re-use returned id's then store them in a `returnedIds` set. The basic logic then becomes:
   - If `returnedIds` has any elements pop first element from set and return it
   - Else new id becomes current_max_id + 1

Share Improve this answer Follow

answered Aug 22, 2017 at 6:33
ChrisWue
20.1k 4 38 104

For the violation of encapsulation, of course those static method will be private. I putted them public for testing purpose. And I think that your last point might be a good solution. Like I said, I think I will never reach the end of the generator during runtime, so I shouldn't bother to reuse old Id – Olivier Aug 22, 2017 at 6:39

Of course, the *set* in your point 4 would typically rather be a *stack* (or *queue* perhaps) – Hagen von Eitzen Aug 22, 2017 at 15:16

@HagenvonEitzen: Yes, most likely – ChrisWue Aug 22, 2017 at 19:33

23

If your unsigned long is large enough, and the rate of ID generation is low enough, you can skip the isIDUsed check entirely and just never reuse an ID. For example, if your unsigned long is 64-bit, you could generate a million IDs per second for almost 600 000 years before you run out:

$$\frac{2^{64}}{1000000 * (365 * 24 * 60 * 60)} \approx 584942$$

Share Improve this answer Follow edited Aug 22, 2017 at 10:09

answered Aug 22, 2017 at 9:15
Fax
331 1 4

it is not about exhausting ALL possible unique numbers. it is about the chance of "drawing" the same number twice consecutively (or at least in a small interval). but, yeah, in 600000 years, the chance of this happening is a big as the chance of seeing God's buttocks while peeing in the bushes. – Gravity Nov 29, 2018 at 14:02

5

Maybe you don't need to **generate** the ID. Would it be enough for you to **have** an ID? This would be unique:

```
public:
    unsigned long getID() const
    {
        return (unsigned long)this;
    }
```

Share Improve this answer Follow

answered Aug 22, 2017 at 14:54
Adamski
67 1

2 It is not fully specified, but may be the case that `swap(A,B)` should swap IDs as well – Hagen von Eitzen Aug 22, 2017 at 15:18

2 Note that while I would not expect this to happen on a practical system, the standard seems to allow for a platform with `sizeof(void*) > sizeof(long)`, since the minimum width for a `long` is 32 bits. Changing the type to `std::size_t` should remove this risk. – IllusiveBrian Aug 22, 2017 at 21:11

5 This is a terrible idea. I used to ask an interview problem about how many different things can go wrong when you do this, and how to fix them without introducing security holes in your application. – Eric Lippert Aug 22, 2017 at 23:55

2 `unsigned long` is implementation-defined, and may have fewer bits than the address (and really does; e. g., Microsoft C++ defines it as 32 bit wide on amd64). C++11 introduces `std::uintptr_t` to be wide enough to hold bits of a pointer. – kkm Aug 23, 2017 at 1:48

2 Can you provide an example @EricLippert? – Olivier Aug 23, 2017 at 3:38

2

I can envision situations in which sharing the memory address of an object would be an unacceptable security hole. I want to offer an alternate solution for such cases. This uses a simple static counter to increment the GUID.

```
class HasGuid {
private:
  static unsigned long currentId = 1;
  unsigned long guid_;
protected:
  HasGuid() : guid_(currentId++) {}
public:
  unsigned long Guid() { return guid_; }
};
```

Any class that needs a GUID can simply inherit from `HasGuid`. The GUID, itself, is safely hidden and can't be changed. And the GUID will be unique across all instances of all classes that inherit from `HasGuid`.

A further option may be to use Adamski's solution. But hash the address with a secret key before returning it.

```
private:
  const unsigned long secretKey_ = {some random string of digits};
public:
  unsigned long getId() {
    return (unsigned long)this ^ secretKey_;
  }
```

Note: I know I'm a sinner for placing my braces on the same line as the function/class name. It's a habit I picked up from JavaScript.

Edit: I recently learned in a software development class that GUIDs are usually just 16 byte random numbers. The odds of two random 16 byte numbers being identical is so fantastically small, that it is safe to assume they will never be identical across any instance of any class in any applications in use across the universe. In fact, to call the odds astronomical would be giving too much credit to astronomy (anecdote stolen from this video by 3Blue1Brown). Also, most operating systems have a built in GUID tyoe that you can generate at any time by calling the appropriate API function.

Share Improve this answer Follow edited Oct 5, 2017 at 8:42 answered Aug 23, 2017 at 3:38

Nathan
121 3

1

First look -

`generateID()` should also `addID()` automatically. This will make c-tor much more simple.

One note for this:

```
static unsigned long generateID() // Generate a valid ID
{
  static unsigned long id = 0;

   while (Foo::isIDUsed(id)) // If all ID are taken, create an infinite loop!
      ++id;

   return id;
}
```

`id` variable is static. This means it it accessed using lock + semaphore or probably is atomic.

Why not move the `id` inside the class?

About this:

```
static bool isIDUsed(unsigned long id)
{
  return s_usedID.count(id) == 1 ? true:false;
}
```

you do not need "? true : false", this is just fine:

```
static bool isIDUsed(unsigned long id)
{
  return s_usedID.count(id) == 1;
}
```

I am not `std::set` expert, but I think if you need to find if value is in set you can use:

```
static bool isIDUsed(unsigned long id)
{
  return s_usedID.find(id) != s_usedID.end();
}
```

Finally, if you have C++11, you can clean up things some more, including you might remove constructor and use default generated constructor.

And really final notes - why not move this ID thing a separate class, then you include it in "user" class via inheritance or via static member. Then the user class will not need to have user destructor at all.

If you have questions, I can give you examples.

Share Improve this answer Follow

answered Aug 22, 2017 at 15:08

Nick
1,326 8 18

-2

As I think it is impossible to have 2 instances of a class to be created in the very same nanosecond, I guess that unique identifiers can be generated using the *chrono* library:

```
 std::chrono::high_resolution_clock m_clock;
 long unique_ID = std::chrono::duration_cast<std::chrono::nanoseconds>
(m_clock.now().time_since_epoch()).count();
```

You can then cast to unsigned long. You may also consider adding a small sleep time.

Share Improve this answer Follow

answered Oct 4, 2019 at 12:46

Dji enne
1 1

1 I'm reasonably sure that even if the initial assumption of "no two classes are created at the same nanosecond" were true (on multicore systems that assumption is tenuous at the very least), the ticksize for a high_resolution_clock is not specified in the standard and therefore may be larger than nanosecond-precision support would require. – Vogel612 Oct 4, 2019 at 13:43